

MÓDULO IV

Introdução à Programação II

MÓDULO IV

Boas práticas de Programação

O que um programador faz?

A vida de um programador não é fácil. A tecnologia é algo que não para de evoluir e a cada dia surge uma forma diferente de escrever um código. A carreira de um profissional de informática é algo de sua responsabilidade.

O código produzido por esse profissional também é de sua responsabilidade. Um bom código não é somente aquele que é funcional, mas também aquele que não tem valores exorbitantes para ser mantido. **A maior parte dos programadores não gostam de alterar códigos mal escritos.** Isso é algo que traz muita frustração e muitas vezes um retrabalho desnecessário.

MÓDULO IV

O QUE É PROGRAMAÇÃO e o que programadores fazem?



O que é Programação?

Escrever instruções muito específicas para uma máquina burra, mas muito obediente.

Quando você pede um café a um humano...

Exemplo:

Quando você pede um café a uma máquina...

A Educação é o primeiro passo para um futuro melhor...

MÓDULO IV



MÓDULO IV

Como um Programa de Computador Funciona?



A Educação é o primeiro passo para um futuro melhor...

MÓDULO IV

A programação é para mim?

Você deve responder, no mínimo, 2-3 "SIM" para essas questões:

1. Você é fascinado por quebra-cabeças lógicos e enigmas?

2. Você está disposto a aprender novas coisas e, na maior parte do tempo, por conta própria?

3. Você tem paciência e resiliência para resolver problemas difíceis?

4. Você é naturalmente curioso sobre como as coisas funcionam?



MÓDULO IV

Então o que fazem os
programadores/desenvolvedores?



1 Desenham, escrevem, testam,
documentam e mantêm o software.

2 Trabalham em equipe o que, normal-
mente, envolve cliente e gestor.

MÓDULO IV

O que os programadores não fazem?

- 1 Arrumam seu computador e sua conexão de internet.
- 2 Invadem o computador do seu amigo.



MÓDULO IV

Quais são as dificuldades e frustrações da programação?



1

Esmurrar a parede é completamente normal. Algumas vezes, seu código não funciona como você achou que funcionaria.



2

De vez em quando haverá pouquíssimo tempo e você terá que trabalhar até tarde para alcançar o prazo.



3

Consertar e manter o código de outras pessoas (se estiver mal escrito).



4

Participar de reuniões desnecessárias.

MÓDULO IV

5

O tempo estimado para o desenvolvimento é regularmente errado por um fator de 2-3, devido a requerimentos vagos de software, problemas inesperados, aumento da complexidade, etc.

O que pensei:
Maratona



Como realmente é:
Corrida de obstáculos



MÓDULO IV

Arrumar bugs misteriosos

- 6 Imagine que você é um engenheiro de software da NASA. Um dia, falam para você arrumar um bug que ocorreu 777 dias atrás, quando Marte e a Terra se alinharam. A causa é desconhecida:

Resolva o bug!

O código que contém o bug foi escrito por um cientista da NASA que se aposentou anos atrás.

Leva 15 minutos para transmitir sinal

Somente ocorre quando a pressão atmosférica é 501,53 Pa e a temperatura é -189,75 F.

Mais 2 horas para se alinharem

Não se preocupe, isso é só uma combinação dos piores cenários de um bug. Programação pode ser frustrante às vezes, mas pode ser bastante recompensadora.

MÓDULO IV

12 boas práticas da programação segundo o livro Código Limpo

Antes de entrarmos em cada um dos 12 itens, é importante ressaltar que aplicar as dicas, principalmente se você não tem muita experiência, pode ser extremamente desafiador. Por isso, vá adotando cada item de modo gradual para evoluir como um desenvolvedor.

Com o tempo, e muita paciência, você estará adotando as práticas com naturalidade. Inclusive, é exatamente essa uma das propostas do livro: adotar o processo de evolução como uma rotina, algo que faremos com naturalidade ao longo do tempo.

Dito isso, vamos conhecer as 12 boas práticas de programação para deixar seu código limpo:

1. Escreva código legível

O código legível é aquele que é fácil de entender à primeira vista. Isso envolve escolher nomes significativos para variáveis, funções e classes, de modo que o propósito e a função sejam claros.

Além disso, o código deve ser formatado de maneira consistente e seguir convenções de estilo para facilitar a leitura e a manutenção. Essa prática pode ser difícil no início de carreira, mas deve ser uma busca constante.

MÓDULO IV

2. Construa funções pequenas e coesas

Funções pequenas e coesas fazem uma única coisa e, geralmente, a fazem bem. Isso torna o código mais fácil de entender, testar e reutilizar. Funções grandes e complexas podem se tornar difíceis de compreender e podem acumular muitas responsabilidades, tornando a manutenção do software problemática.

3. Escolha nomes significativos para o código

Escolher nomes descritivos e significativos para variáveis, funções e classes é fundamental na criação de Código Limpo. Nomes bem escolhidos comunicam o propósito e a função do elemento no código, reduzindo a necessidade de comentários explicativos.

Indo além, eles permitem que outros desenvolvedores entendam o código sem qualquer tipo de problema, o que é ótimo para evitar a perda de tempo e criação de novas falhas.

MÓDULO IV

4. Deixe comentários relevantes

Embora o objetivo seja escrever código autoexplicativo, como falamos, às vezes os comentários são necessários para esclarecer partes complexas ou decisões de design do código. Nesse sentido, os comentários devem explicar o porquê, a intenção ou a lógica por trás do código, em vez de simplesmente repetir o que o código faz. Quanto mais didático, melhor.

5. Use uma formatação consistente

A formatação consistente é vital para a legibilidade do Código Limpo. Manter um estilo de formatação consistente em todo o projeto facilita a leitura e ajuda os desenvolvedores a se concentrarem no conteúdo em vez de se preocuparem com a formatação.

A boa organização encurta o processo de compreensão do código, lembre-se disso.

MÓDULO IV

6. Abuse dos testes unitários

Os testes unitários são parte essencial da garantia de qualidade do software. Eles ajudam a verificar se as partes individuais do código funcionam como deveriam.

Um código com testes bem escritos proporciona confiança aos desenvolvedores para fazer alterações sem medo de introduzir erros. Mas, caso isso aconteça, os testes ajudarão a detectar e eliminar as novas falhas.

7. Adote o Princípio da Responsabilidade Única

O Princípio da Responsabilidade Única prega que uma classe ou função deve ter apenas uma única responsabilidade. Isso facilita a manutenção, pois alterações em uma responsabilidade não afetarão as outras.

Classes e funções com uma única responsabilidade também tendem a ser mais reutilizáveis e, de maneira geral, cooperam para o sistema ser construído com um Código Limpo.

MÓDULO IV

8. Aposte no princípio Aberto/Fechado

Esse princípio sugere que o código deve estar aberto para extensão, mas fechado para modificação.

Isso significa que você pode adicionar novos recursos ou comportamentos através da criação de novas classes ou funções sem alterar o código existente.

Isso ajuda a evitar efeitos colaterais não desejados, muito comuns em processos de desenvolvimento com baixa organização e pouca cobertura de testes automatizados.

9. Garanta o Princípio da Substituição de Liskov

O Princípio da Substituição de Liskov estabelece que as classes derivadas (subclasses) devem ser substituíveis por suas classes base (superclasses) sem afetar a funcionalidade do programa.

Na prática, isso garante que as substituições de classes não quebrem o comportamento esperado do código, ou simplesmente adicionem complexidade de leitura e manutenção para outros devs.

MÓDULO IV

10. Mantenha o Princípio da Inversão de Dependência

Esse princípio defende que módulos de alto nível não devem depender de módulos de baixo nível, mas ambos devem depender de abstrações. Isso promove um design mais flexível e desacopla as partes do sistema, tornando-o mais fácil de modificar e manter.

11. Elimine a duplicação de código

Duplicação de código resulta em manutenção difícil e inconsistências — fatores que são contra as premissas do código limpo. Eliminar duplicações promove a consistência e evita a necessidade de fazer alterações em vários lugares quando requisitos mudam.

E acredite, isso ainda acontece em muitos softwares criados por profissionais com pouca experiência. Então, internalize essa regra e evite cair nessa armadilha.

MÓDULO IV

12. Controle a complexidade

Controlar a complexidade do código é crucial em todos os sentidos. Funções pequenas, nomes significativos, princípios de design sólidos e escolha de abstrações adequadas contribuem para manter a complexidade sob controle.

Isso facilita a manutenção e evita que o código se torne difícil de entender. Além disso, vai reduzir a chance de bugs inesperados aparecerem. Portanto, toda a complexidade deve ser abolida do sistema.

Escrever código limpo é um processo

Lembre-se que as dicas trazidas ao longo deste conteúdo não vão, automaticamente, te tornar um desenvolvedor sênior, mas certamente te colocarão no caminho para tal função.

Muito além do cargo, um desenvolvedor sênior é aquele que tem condições de escrever um código fácil de compreender, sem complexidades e que não atrapalhe a evolução do sistema.

Portanto, adotar a cultura do Código Limpo faz parte das boas práticas de programação que um profissional do desenvolvimento precisa ter.

MÓDULO IV

Clean Code

Um código limpo deve ser:

- **Simples:** código fácil de entender;
- **Direto:** vai direto ao ponto, não dá “voltas” para atingir seu objetivo;
- **Eficiente:** código que faz o que é proposto;
- **Sem duplicidade:** não faz o que outra parte do código já faz;
- **Elegante:** porque é diferente dos outros códigos;
- **Feito com cuidado:** quem fez teve preocupação em produzir aquele código.

Antes de falarmos sobre como fazer para atingir esse nível de qualidade, vamos falar um pouco sobre testes.

MÓDULO IV

Escrevendo um bom código

Nomes significativos

Métodos, nomes de variáveis e etc. devem possuir nomes que significam alguma coisa em relação ao seu objetivo. Os nomes utilizados devem responder todas as questões a seguir:

- *Porque existem?*
- *O que fazem?*
- *Como são usadas?*

Vamos imaginar que um sistema de um motor de um carro tenha um **método** com o nome de “**run**” ao invés de “**acelerar**”. Se você pegar um código com esse nome você terá que estudar todo o método para saber o que ele faz.

MÓDULO IV

Classes e métodos

Nome de **classes** devem ser substantivos e não conter verbos.

Já nomes de **métodos** devem conter verbos pois eles indicam ações.

A regra para **métodos** é: “A primeira regra dos **métodos** é que eles devem ser pequenos.

A segunda regra é que eles devem ser menores ainda.”

Métodos e **classes** menores são mais fáceis de ler e entender, além de manter é claro.

Segundo o livro, podemos considerar as seguintes métricas:

- Métodos ≤ 20 linhas;
- Linha ≤ 100 caracteres;
- Classe = 200 a 500 linhas.

MÓDULO IV

Claro que toda regra tem sua exceção. Se você tem uma **classe** que vai precisar de mais linhas, um **método** que também precise de mais linhas, isso não é um problema.

*“**Métodos e funções** devem fazer somente uma coisa, fazê-la certa e somente fazê-la”.*

Poderíamos analisar essa frase como um princípio da coesão no seu código.

Muitas vezes não é fácil saber se aquele **método** está fazendo somente uma coisa.

Uma dica para isso é: você deve tentar extrair parte do seu código para um **método**, se você conseguir é porque aquele seu método realmente não está tendo uma função apenas.

MÓDULO IV

Comentários nos códigos

Comentários, apesar de importantes, podem trazer desinformação.

Se um método ou uma classe estiver bem escrito, a importância do comentário é minimizada.

Outro ponto importante, um comentário não irá esconder um código ruim. Observe o exemplo a seguir:

```
Date d1;
```

Esse código está ruim, não adianta mudarmos para:

```
//dia da semana
```

```
Date d1;
```

Esse comentário não irá se propagar para todo o código e sempre que você se deparar com uma linha como “`d1.after(d2);`” você vai continuar não entendendo o propósito do código.

Podemos tentar colocar uma regra nisso. Muitas vezes quando se comenta um código, pode ser que o mesmo precise ser refatorado. Lembra dos exemplos anteriores onde “**d1**” passou a ser “**dataCompra**”? Com essa mudança seu código pode ser entendido por todos e se fizermos essa refatoração o código passa a não precisar mais de comentário.

MÓDULO IV

Importância dos testes

Construir um software não é somente escrever código e vê-lo funcionar, é você saber que aquele código será manutenível e que outras pessoas vão alterá-lo.

Para isso, o **teste** é fundamental!

Você tem que ser responsável por aquilo que escreve e saber que seu sistema tem que continuar funcionando.

Neste contexto, temos a primeira dica sobre um código limpo:

Toda linha que você escrever deve estar **testada** e ponto final !

Muitas empresas veem **testes** como gastos maiores no projeto, o que de fato acontece, porém a qualidade do software produzido é algo significativo.

Quando não se produz **teste** automatizado, a quantidade de **testes** manuais são maiores e muitas vezes o custo desses **testes** também é maior.

MÓDULO IV

Formatação

“Formatação é importante, pois se trata de comunicação.”

Temos que considerar que o código é a maneira que a equipe de desenvolvimento vai se comunicar. Uma pessoa não gostaria de receber uma carta cifrada onde tivesse que interpretar o que está escrito nela, podemos pensar assim na hora de escrever um código.

Outra ponto importante é que se você pega um código bem estruturado, você vai querer mantê-lo bem estruturado. É ruim para qualquer desenvolvedor ter acesso a um código sem formatação, sem endentação e ter que fazer sua leitura como se fosse um texto sem qualquer pontuação.

Além disso, métodos com conceitos relacionados devem ficar verticalmente próximos e a ordem dos métodos deve criar um fluxo de leitura melhorando a legibilidade do código.

Uma boa endentação é fundamental, mas não podemos ter muitos níveis.

MÓDULO IV

Tratamento de erros

"Quando estamos programando devemos tratar os possíveis erros que nossa aplicação poderá lançar, as coisas podem dar errado e temos que estar certos que nosso código fará o que deve fazer."

Tratamento de erro é de responsabilidade do desenvolvedor. É preciso garantir que o código vai ter um tratamento para cada situação. Prefira lançar uma *exception* ao invés de retornar um código de erro.

Estes retornos desorganizam a chamada do método e pode-se facilmente esquecer de verificá-los.

Dentro do seu método você já pode ver o erro que está sendo retornado e tratá-lo ali.

Defina o fluxo do método separando as regras de negócio de erros ou outras situações.

Para seus erros, crie mensagens informativas mencionando a operação que falhou e o tipo de falha.

Procure utilizar exceptions para situações inesperadas, por exemplo: seu código está lendo um arquivo e a rede se tornou indisponível.

MÓDULO IV

Entendendo as exceções

As exceções ocorrem quando algo imprevisto acontece, elas podem ser provenientes de erros de lógica ou acesso a recursos que talvez não estejam disponíveis.

Alguns possíveis motivos externos para ocorrer uma exceção são:

- Tentar abrir um arquivo que não existe.
- Tentar fazer consulta a um banco de dados que não está disponível.
- Tentar escrever algo em um arquivo sobre o qual não se tem permissão de escrita.
- Tentar conectar em servidor inexistente.

Alguns possíveis erros de lógica para ocorrer uma exceção são:

- Tentar manipular um objeto que está com o valor nulo.
- Dividir um número por zero.
- Tentar manipular um tipo de dado como se fosse outro.
- Tentar utilizar um método ou classe não existentes.

MÓDULO IV

O TDD

O TDD (*Test Driven Development / Desenvolvimento orientado a teste*) é parte da metodologia XP e também utilizado em diversas outras metodologias, além de poder ser utilizada livremente.

O TDD transforma o desenvolvimento, pois deve-se primeiro escrever os testes, antes de implementar o sistema. Os testes são utilizados para facilitar no entendimento do projeto, pois clareiam a ideia em relação ao que se deseja em relação ao código. Segundo Presmann (2011), "os componentes individuais são testados para garantir que operem corretamente. Cada componente é testado independentemente, sem os outros componentes de sistema. Os componentes podem ser entidades simples, tais como funções ou classes de objetos, ou podem ser grupos coerentes dessas entidades".

Um sistema é um conjunto de unidades integradas, por este motivo é importante os testes unitários para ver se no micromundo tudo funciona, mas também temos de testar a integração, ou seja, ao integrar dois ou mais componentes, devemos realizar testes para verificar se a integração funciona. Voltemos ao exemplo do carro, não adianta eu testar a roda, a porta, o volante, mas não testar o carro completo. Erros podem ocorrer, justamente no processo de montagem/integração de componentes.

MÓDULO IV

TDD nada mais é que o desenvolvimento guiado por testes.

As três regras do TDD são:

- Você não pode escrever um código até que tenha criado um teste falhando;
- Você não pode escrever mais teste do que seja suficiente para falhar;
- Você não pode escrever mais código do que o suficiente para passar no teste que está falhando.

Assim, se você tiver que testar se um **CPF** é válido, por exemplo, você deve criar alguns testes tais como:

- se o **CPF** for em branco;
- se o **CPF** estiver com menos caracteres.

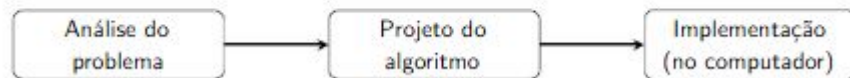
Os testes devem considerar as características F.I.R.S.T:

- ***F** (*Fast*): deve ser rápido. Testes longos tiram a motivação dos profissionais responsáveis por sua execução
- ***I** (*Independent*): não podem depender um do outro pois se um falha o outro vai falhar também;
- ***R** (*Repeatable*): executando mais de uma vez eles devem retornar sempre o mesmo resultado;
- ***S** (*Self-Validating*): devem se autovalidar;
- ***T** (*Timely*): devem ser feitos antes do código.

MÓDULO IV

Resolução de Problemas com o Computador Algoritmos e Programação de Computadores

Análise do Problema



Definição clara do que se se deseja resolver.

- Qual é a entrada do problema?
- Qual é a saída desejada?
- Como transformar a entrada na saída?
- É necessário realizar algo durante a transformação?
- etc.

MÓDULO IV

1. Entendendo o problema.

- O que é desconhecido?
- Quais são os dados conhecidos?
- Estes dados são suficientes para resolver a questão?

2. Elabore um plano.

- Qual a ligação entre o conhecido e o desconhecido?
- Se não há uma ligação direta entre eles, há algum problema relacionado que pode ser útil na resolução?
- Se sim, este conhecimento é útil para o problema em questão?
- Se não, o problema pode ser apresentado de outra forma?

MÓDULO IV

3. Execute o plano.

- Verifique cada passo do plano.
- Está claro que o passo está correto?
- É possível provar que o passo está correto?

4. Examine a solução.

- É possível verificar o resultado?
- É possível obter o resultado de outra forma?
- É possível utilizar a solução ou o método em outro problema?

MÓDULO IV

Algoritmo

Sequência finita de instruções para executar uma tarefa.

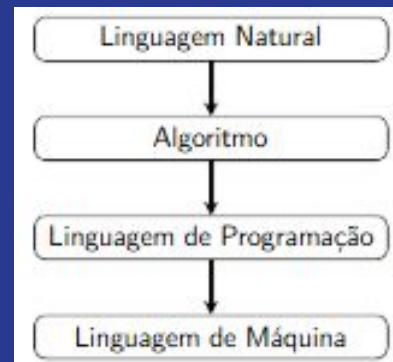
- Bem definidas e não ambíguas.
- Executáveis com uma quantidade de esforço finita.
- Executáveis em um período de tempo finito.

Bolo simples

- 1 - tigela ← ingredientes
- 2 - bater (tigela) - durante (5 min)
- 3 - untar (forma)
- 4 - forma ← conteúdo (tigela)
- 5 - colocar (forma, fogão)
- 6 - esperar (40 min)
- 7 - esperar esfriar

Um algoritmo descreve um padrão de comportamento com:

- instruções sequenciais;
- bifurcações;
- repetição



MÓDULO IV

“Faça funcionar antes de fazer funcionar rápido.”

Existem diversas formas de representar de um algoritmo.

- Descrição narrativa
- Diagramas (fluxograma)
- Pseudocódigo
- Linguagem de programação
- Linguagem de máquina
- etc.

Não existe um consenso de qual é a melhor forma, mas o algoritmo deve ser representado da forma mais clara possível.

MÓDULO IV

A construção do algoritmo faz parte do planejamento e execução do plano, conforme as seguintes etapas:

1 - Definição dos dados a serem manipulados:

- Quais informações serão necessárias? (*variáveis/ constantes*)
- Quais os tipos de cada? (*string/ number*)

2 - Obtenção destes dados (*entradas*).

3 - Definição da sequência de execução dos processos.

4 - Exibição dos resultados (*saídas*).

MÓDULO IV

Um algoritmo computacional será executado por um programa de computador, sendo uma sequência de instruções que vai manipular os dados.

Instruções: comandos que determinam a forma pela qual os dados devem ser tratados.

Dados: informações recolhidas/fornecidas por diversos meios e que serão processadas pelo computador através das instruções.

“Conhecimento não tem valor a não ser que seja posto em prática.”

MÓDULO IV

Exercícios

- Faça a descrição narrativa do algoritmo para trocar a lâmpada queimada de um abajur.
- Faça a descrição narrativa do algoritmo para ler a idade de uma pessoa (em **anos**, **meses** e **dias**), e escrever a quantidade de **horas** vividas.
- Monte um fluxograma para um algoritmo que leia as 3 notas de um aluno e calcule a média final desse aluno. Considerar que a média é ponderada e que os pesos das notas são: 70, 85 e 90, respectivamente. Defina o pseudocódigo do algoritmo gerado.

MÓDULO IV

EXERCÍCIOS...

MÓDULO IV

FIM...